Image Processing ASIC

Full RTL-to-GDSII Design Flow

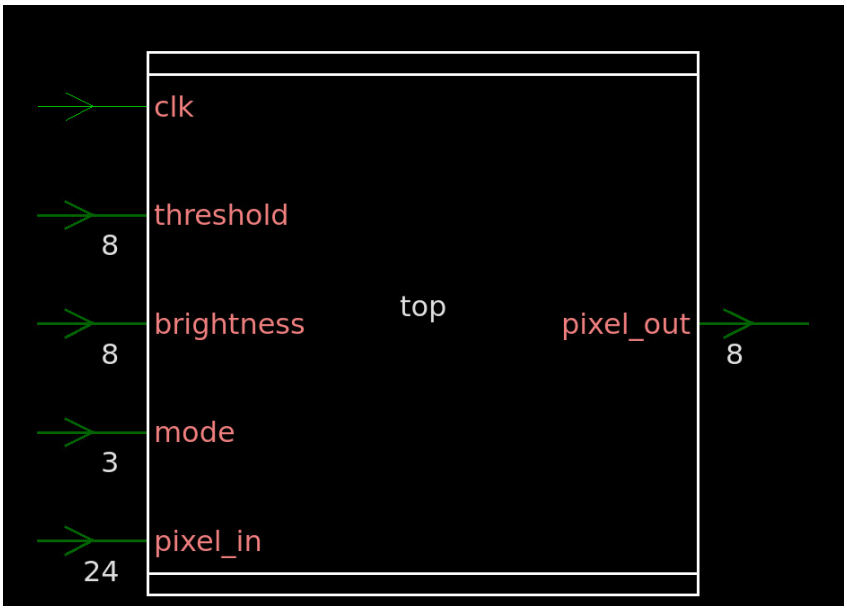
1. Project Overview

Design Objectives

- Convert 24-bit RGB pixel to 8-bit grayscale
- Apply a selected image processing operation per pixel
- Implement in Verilog and verify the waveform generated using Testbench
- Run a complete RTL → Simulation →Synthesis →Floor plan →Power plan →Placement →CTS→Route

Design	Image Processing Pipeline
Technology	SAED32nm (32nm Educational PDK)
Image size	64 × 64 pixels (4,096 pixels)
Data format	24-bit RGB input -> 8-bit grayscale output
Tools used	VCS, Verdi, Design Compiler, ICC2
PDK Library	saed32rvt / lvt / hvt standard cells

System Architecture



Top-level port interface of the 'top' module - clk, mode[2:0], brightness[7:0], threshold[7:0], pixel_in[23:0], pixel_out[7:0]

The design is a three-module hierarchy:

- **top** - Unpacks 24-bit pixel_in into R[7:0], G[7:0], B[7:0]. Instantiates grayscale and operation submodules. Routes pixel_out.
- **grayscale** - Pure combinational. Computes: gray = (R + G + B) / 3
- **operation** - Combinational always @(*) block. Selects output based on mode[2:0].

Operating Modes

Mode	Name	RTL Logic	Effect
`3'b000`	**Pass through**	`out = gray`	Raw grayscale output - no change
`3'b001`	**Invert**	`out = 255 - gray`	Photographic negative of grayscale image
`3'b010`	**Brightness**	`out = gray + brightness`	Adds brightness to the pixels
`3'b011`	**Threshold**	`out = (gray > threshold) ? 255 : 0`	Binary segmentation - black or white only

```
module top(
    input clk,
    input [23:0] pixel_in,
    input [2:0] mode,
    input [7:0] brightness,
    input [7:0] threshold,
    output [7:0] pixel_out
);
wire [7:0] R, G, B;
wire [7:0] gray;

assign R = pixel_in[23:16];
assign G = pixel_in[15:8];
assign B = pixel_in[7:0];

grayscale u1 (.R(R), .G(G), .B(B), .gray(gray));

operation u2 (
    .gray(gray),
    .mode(mode),
    .brightness(brightness),
    .threshold(threshold),
    .out(pixel_out)
);
endmodule
-----------------------------------------------------------------------------------------------------------------------------------------------------------
module operation(
    input  [7:0] gray,
    input  [2:0] mode,
    input  [7:0] brightness,
    input  [7:0] threshold,
    output reg [7:0] out
);
always @(*) begin
    case(mode)
        3'b000: out = gray;
        3'b001: out = 8'd255 - gray;
        3'b010: out = gray + brightness;
        3'b011: out = (gray > threshold) ? 8'd255 : 8'd0;
        default: out = gray;
    endcase
end
endmodule

module grayscale(
    input  [7:0] R, G, B,
    output [7:0] gray);   endmodule
```

2. Simulation - VCS + Verdi

2.1 Simulation Setup

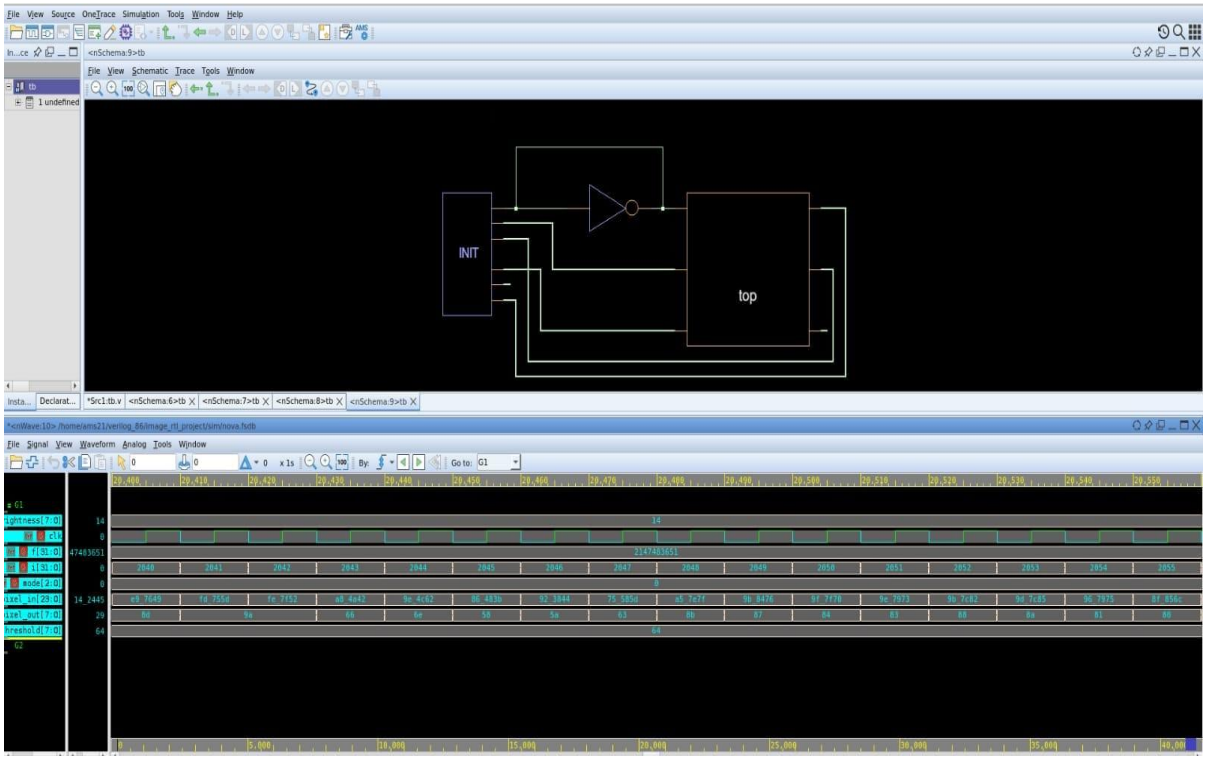
- Tool: Synopsys VCS (Verilog Compilation Simulator)
- Waveform viewer: Synopsys Verdi (FSDB format)
- Testbench file: tb.v
- Image data: 64×64 pixels stored as hex values in image.hex (4,096 lines)
- Output: Processed pixels written to out.hex via $fwrite

Testbench Operation - Step by Step

- $readmemh loads all 4,096 pixels from image.hex into a reg array
- For each pixel: drive pixel_in, wait #10 (one clock), capture pixel_out
- Each pixel_out written to out.hex for post-processing analysis
- $display prints IN= and OUT= for every pixel which would be visible in terminal
- $fsdbDumpvars dumps all signals to nova.fsdb for Verdi inspection
- Simulation ends with $finish after all pixels processed.

```
module tb;
reg clk;
reg [23:0] pixel_in;
reg [2:0] mode;
reg [7:0] brightness;
reg [7:0] threshold;
wire [7:0] pixel_out;
reg [23:0] mem [0:4095];  // for 64x64
integer i;
integer f;
top uut (
    .clk(clk),
    .pixel_in(pixel_in),
    .mode(mode),
    .brightness(brightness),
    .threshold(threshold),
    .pixel_out(pixel_out)
);
always #5 clk = ~clk;
initial begin
    $fsdbDumpfile("nova.fsdb");
    $fsdbDumpvars(0, tb);
end
initial begin
    clk = 0;
    $readmemh("../data/image.hex", mem);  // Read image
    // Set parameters
    mode = 3'b000; // grayscale
    brightness = 20;
    threshold = 100;
    f = $fopen("../data/out.hex", "w");  // Output file
    for(i = 0; i < 4096; i = i + 1) begin
        pixel_in = mem[i];   #10;
        $fwrite(f, "%h\n", pixel_out);
        $display("IN=%h OUT=%h", pixel_in, pixel_out);  end
    $fclose(f);
    $finish;
    end
endmodule
```

2.2 Simulation Results



- All 4,096 pixels processed correctly and output verified against expected grayscale formula
- pixel_out responds within the same clock cycle which confirms purely combinational path
- Waveform shows pixel_in changing every 10 ns; pixel_out tracks immediately with zero latency
- brightness[7:0] = 14, threshold[7:0] = 64 visible in waveform - static control inputs
- mode[2:0] stays at 0 throughout & grayscale pass through verified as baseline
- f[51:0] counter in Verdi confirms 4,096 simulation steps executed
- FSDB file (nova.fsdb) captures complete signal history for post-simulation debug

> **Key Observation**
> The design is purely combinational so no pipeline registers. The clock in the testbench is used only for sequencing stimulus, not for the datapath logic. pixel_out is valid within one combinational propagation delay after pixel_in changes.

3. Synthesis - Design Compiler

3.1 Synthesis Flow

- Tool: Synopsys Design Compiler (dc_shell)
- Target library: saed32rvt_ss0p75vn40c.db - RVT cells, slow-slow corner, 0.75V, - 40°C
- Link library includes LVT and HVT cells for multi-voltage optimization
- RTL files analyzed individually: grayscale.v, operation.v, top.v
- Design elaborated and linked before constraint application

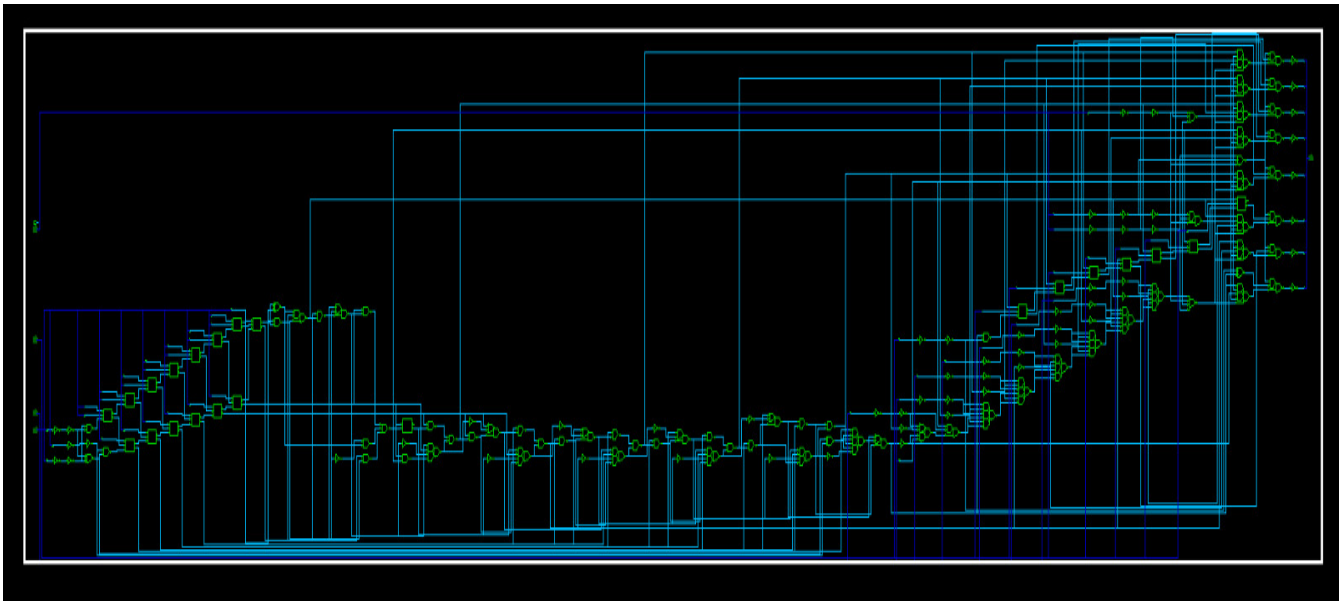
Synthesis Steps Executed

- Step 1 : read_file + elaborate top + link: RTL loaded, design hierarchy confirmed
- Step 2 : compile (initial): First-pass gate-level mapping to SAED32 cells
- Step 3 : Constraints applied: clock, input/output delays, load, transition limits
- Step 4 : compile_ultra: Timing-driven re-synthesis with area recovery
- Step 5 : compile_ultra -incremental: Final timing closure pass
- Step 6 : Outputs: top.mapped.v (gate netlist), top.sdc (constraints), top.ddc

Timing Constraints Applied

- Clock: create_clock -period 0.5 [get_ports clk] → 2 GHz target
- Clock latency: 0.4 ns (models clock tree insertion delay)
- Clock uncertainty: 0.1 ns (models jitter and skew)
- Input delay (max): 1.0 ns | Input delay (min): 0.1 ns
- Output delay (max): 0.5 ns | Output delay (min): −0.1 ns
- Output load: 25 fF on all outputs
- Input transition: 0.3 ns on all inputs
- Max transition (clock): 0.25 ns | Max transition (design): 0.5 ns

3.2 Synthesis Results



Design Compiler gate-level schematic - combinational logic tree from pixel_in/mode inputs to pixel_out, showing MUX/adder/comparator cells synthesized from RTL

- Gate-level netlist generated: top_final.v : instantiates SAED32 standard cells
- Schematic shows: adder tree for grayscale computation, MUX chain for mode selection, comparator for threshold operation
- No flip-flops in the synthesized design : confirms fully combinational datapath
- Reports generated: report_timing, report_area, report_power, report_qor, report_constraint -all_violators
- False paths set on primary I/O to isolate internal combinational path analysis
- SDC written to top.sdc for handoff to ICC2

4. Physical Design - ICC2

4.1 ICC2 Flow Overview

The mapped netlist from Design Compiler (top.mapped.v) is imported into Synopsys IC Compiler II. After synthesis, the design exists only as a **logical netlist** (connections between gates).
ICC2 converts this into a **physical silicon layout**

- Library created: IMG_LIB using saed32rvt_c.ndm (NDM reference library)
- Block imported: top is linked and checked with check_design -checks dp_pre_floorplan
- Scenario: func::nom - functional mode, nominal corner
- Parasitics: TLU+ files loaded (Cmax and Cmin) for accurate RC during placement
- SDC loaded from DC output: top.sdc - same constraints carried through

4.2 Floor planning

- Defines chip structure: core area, boundaries, routing space
- Determines overall layout quality and routing feasibility.
- Pin placement was varied across different sides of the chip. This directly affects how signals enter and leave the design. Poor pin placement can lead to longer routing paths, increased delay, and congestion.

4.3 Power Delivery Network (PDN)

- Power nets created: VDD (power) and VSS (ground)
- connect_pg_net -all_blocks -automatic : connects all standard cell pg pins
- Ring: core_ring_pattern : M7 horizontal, M8 vertical, width=0.4, spacing=0.3, offset=0.5
- Mesh: M6 vertical + M7 horizontal + M8 vertical : pitch=5, interleaving spacing, VDD/VSS alternating
- Rails: M1 std_cell_rail pattern, rail_width=0.06 : standard cell power straps
- All compiled via compile_pg -strategies

4.4 Placement

- place_opt : timing-driven global placement
- legalize_placement : resolves overlaps, aligns to rows
- place_pins -self : automatic pin assignment
- Saved as block : top_placement
- check_design -checks pre_clock_tree_stage run before CTS

4.5 Clock Tree Synthesis (CTS with CCD)

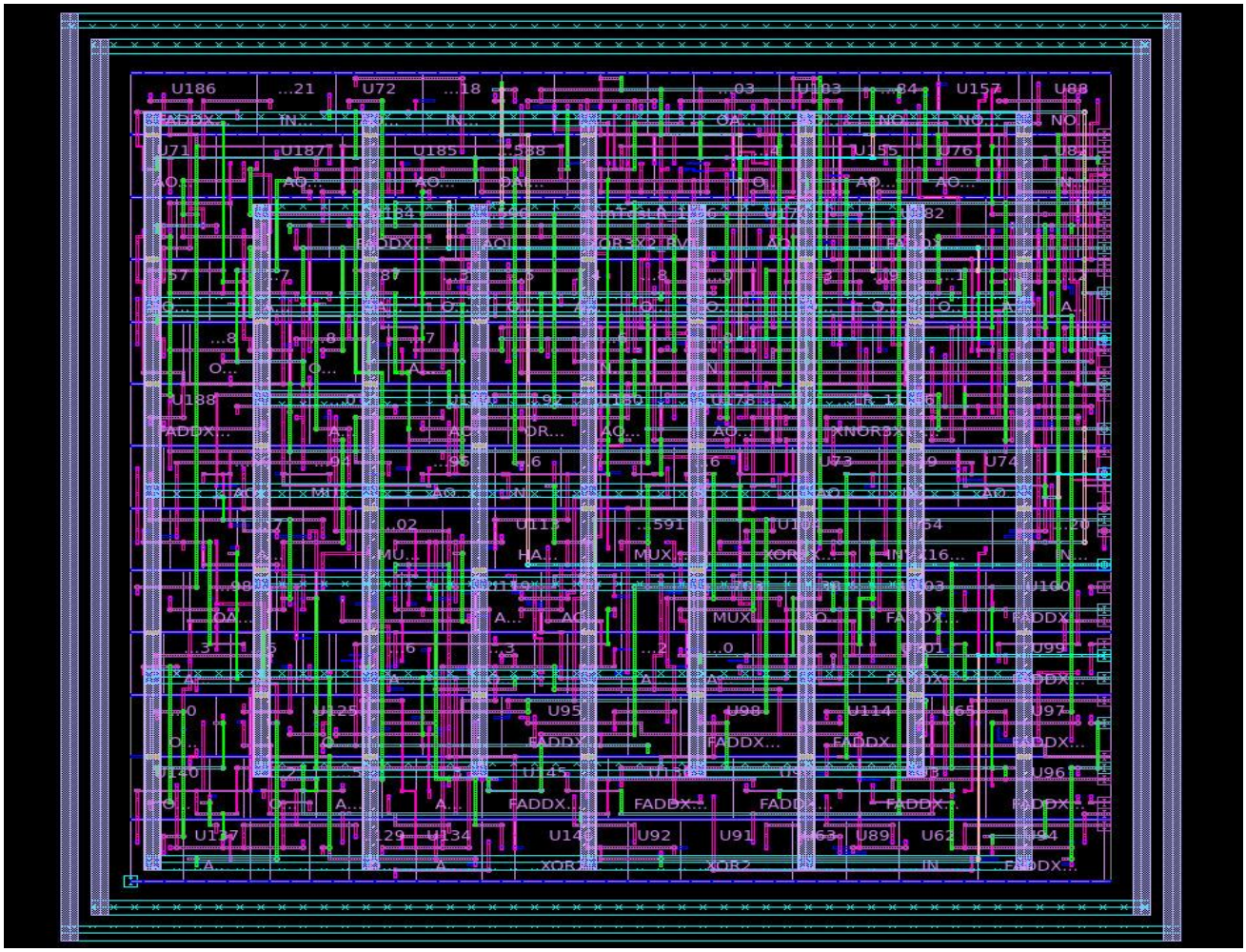
- create_clock -period 1 [get_ports clk] : 1 ns clock for physical timing
- synthesize_clock_tree : initial CTS to build the tree
- CCD (Concurrent Clock and Data) optimization enabled:
  - cts.optimize.enable_local_skew = true
  - cts.compile.enable_local_skew = true
  - clock_opt.flow.enable_ccd = true
- clock_opt -to build_clock : builds the complete clock tree

- clock_opt -from route_clock -to route_clock : routes clock nets
- Saved as block: top_cts_CCD

4.6 Routing

- route_global : timing-driven global routing (crosstalk: false at this stage)
- route_track : timing + crosstalk-driven track assignment
- route_detail : detail routing with antenna fixing:
  - Antenna fixing enabled: route.detail.antenna = true
  - Strategy: use_diodes (ANTENNA_RVT cells inserted automatically)
  - route.detail.timing_driven = true
- route_opt : post-route optimization for final timing closure
- Outputs: top.routed.v, top.routed.sdc, top_func::nom.spef (parasitics)

4.7 Physical Design Results



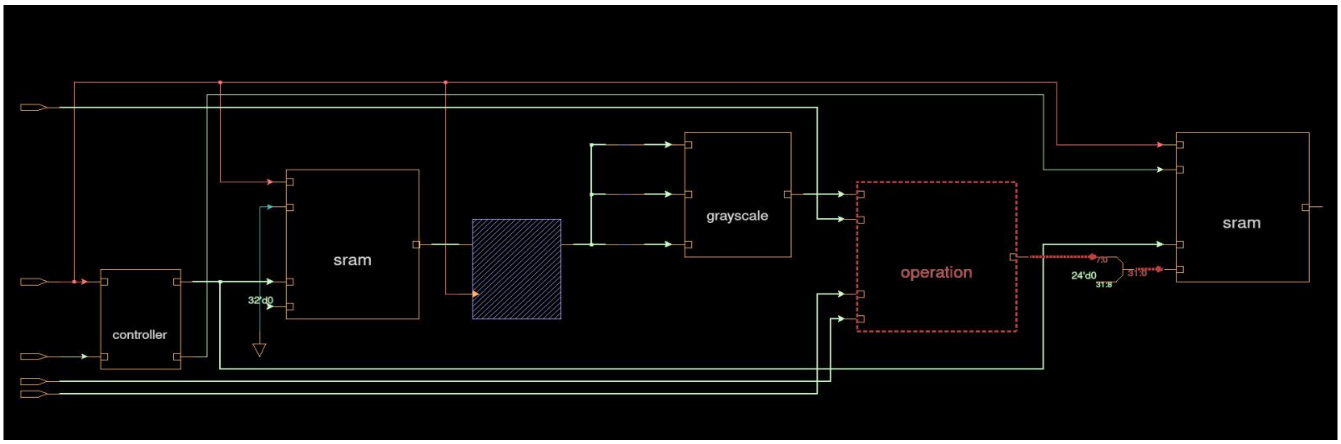
ICC2 routed layout - standard cells placed across the die, M7/M8 power mesh visible as vertical/horizontal metal stripes, detailed routing of signal nets in lower layers

- Full placement and routing completed & no unrouted nets
- Power mesh clearly visible: M7/M8 VDD/VSS rings and mesh straps
- Standard cells densely placed across the core area
- Signal nets routed in lower metal layers (M1- M6)
- Antenna violations corrected via ANTENNA_RVT diode insertion
- SPEF extracted for post-route timing signoff

5. SRAM Integration - Attempted

5.1 What Was Attempted

An enhanced version of the design was prototyped that included on-chip image storage using SRAM, eliminating the need for a host computer to feed pixels one at a time.



SRAM-integrated architecture in Verdi, input SRAM, grayscale, operation, output SRAM modules interconnected

Proposed Architecture

- controller : FSM to sequence read address, processing, and write address
- Input SRAM : stores incoming 24-bit RGB pixels (4096 locations)
- grayscale : same module as current design
- operation : same module, but connected via SRAM bus
- Output SRAM : stores processed 8-bit pixel results

5.2 Problem Faced

> **Root Problem: No SRAM Macro Available**
> Without a compiled SRAM macro from a memory compiler, any attempt to model SRAM using behavioural Verilog (reg arrays) results in the synthesizer mapping each bit to a flip-flop. A 4096-location × 24-bit input SRAM = 98,304 flip-flops. This is structurally impossible to synthesize and place-and-route with a standard cell library.

- Behavioural reg arrays (reg [23:0] mem [0:4095]) are fine in simulation but cannot be synthesized to real SRAM
- Design Compiler maps behavioural memory to flip-flops resulting in large number of FFs for input SRAM alone
- This exceeds the practical area and timing limits of the SAED32 academic PDK
- A real SRAM macro requires a dedicated memory compiler (e.g., ARM Artisan, foundry-specific tool)
- Academic Synopsys licenses do not include an SRAM memory compiler
- Decision made: remove SRAM, keep focus on demonstrating the full RTL-to-routed flow cleanly

> **What This Means**
> The current design correctly demonstrates every stage of the ASIC flow. The SRAM integration remains a future enhancement that requires either an SRAM macro library or a replacement approach (e.g., loading pixels via UART into a register file of practical size).

6. Future Enhancements

Enhancement	Notes
SRAM macro integration	Requires a memory compiler to generate SRAM macros. Not available in academic Synopsys license.
UART image loader	Depends on SRAM availability. UART RX + baud-rate divider + byte-to-pixel state machine would be needed.

7. Tools, Files & Project Structure

Tool Flow

Tool	Stage	Purpose
Synopsys VCS	RTL Simulation	Compile and simulate Verilog RTL; generate hex output file
Synopsys Verdi	Debug / Waveform	View FSDB waveforms; inspect signal values and timing relationships
Design Compiler	Logic Synthesis	Map RTL to SAED32 standard cells; generate gate netlist and SDC
IC Compiler II	Physical Design	Floorplan, Power plan, placement, CTS, routing